El montículo máximo (Max-heap): SplMaxHeap

Caso práctico

La tienda de **BK Programación** organiza una **subasta online** de productos reacondicionados. Cada segundo llegan pujas y la página debe mostrar siempre la **más alta**. Además, el comparador de envíos tiene que encontrar las **3 tarifas más baratas** entre cientos de transportistas. **María** le explica a **Carlos** que el primer problema se resuelve con un **montículo máximo**... y el segundo, curiosamente, también.

1.- Qué es un montículo máximo

Un **montículo máximo** es un árbol binario casi completo en el que cada nodo es **mayor o igual que sus hijos**: el **máximo está siempre en la raíz**. Es el simétrico del montículo mínimo, con los mismos costes:

Operación	En SplMaxHeap	Coste
Añadir	`insert($x)`	O(log n)
Sacar el máximo	`extract()`	O(log n)
Ver el máximo	`top()`	O(1)
Tamaño / ¿vacío?	`count($h)` / `isEmpty()`	O(1)

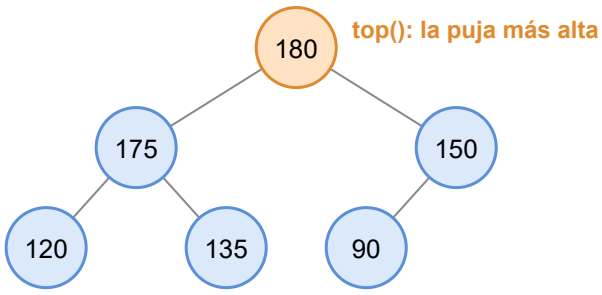


Figura 1. Un montículo máximo: cada padre es mayor o igual que sus hijos

2.- La clase SplMaxHeap

```php
<?php
$pujas = new SplMaxHeap();
foreach ([120, 150, 135, 180, 175] as $puja) {
    $pujas->insert($puja);
    echo "La puja más alta es {$pujas->top()} €", PHP_EOL;   // 120, 150, 150, 180, 180
}
echo $pujas->extract();         // 180: gana la subasta
echo $pujas->top();             // 175
```

Funciona exactamente igual que `SplMinHeap` (las dos heredan de `SplHeap`): mismos métodos, `RuntimeException` si está vacío, y `foreach` **lo vacía**. El recorrido manual muestra por qué: `current()` es lo mismo que `top()` y `next()` **extrae** la raíz.

Desempates. Para ordenar por un valor y desempatar, se insertan arrays: `[puntos, -orden, nombre]` saca primero los más puntos y, a igual puntuación, el que se presentó antes.

> **Debes conocer**
>
> `SplMaxHeap`, `SplPriorityQueue` y un `SplMinHeap` con los valores negados resuelven el mismo problema ("dame siempre el mayor"). Elige `SplPriorityQueue` cuando el dato y su prioridad son cosas distintas (una tarea y su urgencia), y un montículo cuando el propio valor (o un array `[valor, dato]`) decide el orden.

3.- Aplicaciones típicas

Problema	Cómo
El máximo de un flujo de datos (puja más alta, mejor puntuación)	`top()` siempre da el mayor
Los K menores (los vuelos más baratos, los hoteles más cercanos)	Un max-heap de tamaño K: su raíz es el peor de los K mejores
Rankings y podios	`[puntos, -orden, nombre]` y extraer los primeros
La mediana en tiempo real	Un max-heap con la mitad inferior y un min-heap con la superior
Planificar empezando por el trabajo más largo (LPT)	Max-heap de trabajos + min-heap de máquinas

La mediana con dos montículos es un clásico: la mitad inferior de los datos va en un montículo **máximo** (su raíz es el mayor de los pequeños) y la mitad superior en uno **mínimo** (su raíz es el menor de los grandes). Si se mantienen equilibrados, la mediana está siempre en las raíces, y cada dato nuevo cuesta O(log n). A diferencia de la media, la mediana no se deja engañar por un valor extremo (un ático de 3200 € entre pisos de 700 €).

4.- Ejercicio resuelto: la mediana de los alquileres

Ejercicio resuelto

Una web de alquileres muestra el precio **mediano** de los pisos a medida que se publican anuncios.

```
<?php
final class Mediana
{
    private SplMaxHeap $inferior;           // la mitad pequeña: su top() es el mayor de
ellos
    private SplMinHeap $superior;           // la mitad grande: su top() es el menor de
ellos

    public function __construct()
    {
        $this->inferior = new SplMaxHeap();
        $this->superior = new SplMinHeap();
    }

    public function anadir(float $x): void
    {
        if ($this->inferior->isEmpty() || $x <= $this->inferior->top()) {
            $this->inferior->insert($x);
        } else {
            $this->superior->insert($x);
        }
        if (count($this->inferior) > count($this->superior) + 1) {      // reequilibrar
            $this->superior->insert($this->inferior->extract());
        } elseif (count($this->superior) > count($this->inferior)) {
            $this->inferior->insert($this->superior->extract());
        }
    }

    public function valor(): float
    {
        return count($this->inferior) > count($this->superior)
            ? $this->inferior->top()                                    // número impar
de datos
            : ($this->inferior->top() + $this->superior->top()) / 2;       // número par
    }
}
$mediana = new Mediana();
foreach ([650, 720, 590, 3200, 700, 680] as $alquiler) {
    $mediana->anadir($alquiler);
    echo "$alquiler -> mediana {$mediana->valor()}", PHP_EOL;
}
```

```
// 650 -> 650, 720 -> 685, 590 -> 650, 3200 -> 685, 700 -> 700, 680 -> 690
// (la media sería 1090 €: el ático la dispara)
```

Autoevaluación

- ¿Qué devuelve `top()` en un `SplMaxHeap` con 4, 9 y 2? **9.**
- ¿Qué tipo de montículo usarías para quedarte con los 5 precios más baratos de un flujo? **Uno máximo de tamaño 5.**
- En la mediana con dos montículos, ¿qué tipo de montículo guarda la mitad inferior? **Uno máximo.**
- ¿Por qué `next()` en un `SplMaxHeap` hace que disminuya `count()`? **Porque extrae la raíz.**

5.- Ejercicios

Fichero	Contenido
`Ejercicios MaxHeap/e01-SplMaxHeap.php`	La API: `insert`, `top`, `extract`, `valid` / `current` / `next`, rankings con desempate, comparación con `SplMinHeap` y `SplPriorityQueue`
`Ejercicios MaxHeap/e02-SplMaxHeap.php`	Aplicaciones: los K más baratos, la mediana en tiempo real, el trabajo más largo primero
`Ejercicios MaxHeap/problemas/`	`e01` y `e02`: problemas y soluciones